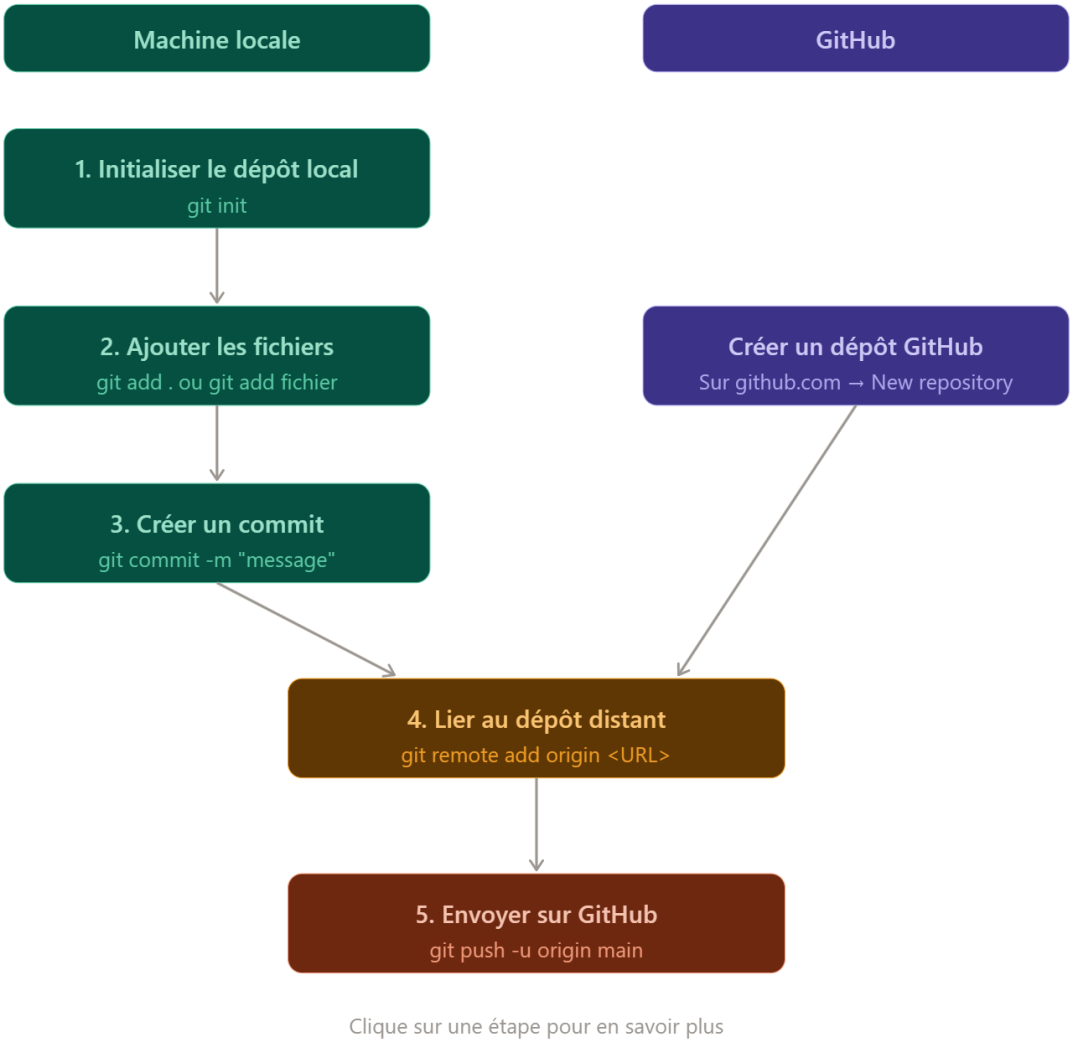
git et repository

Comment lier un fichier a un repository github



Prérequis

- Git installé (`git --version` pour vérifier)
- Un compte GitHub

- Un dépôt créé sur github.com (bouton **New repository**, sans README pour l'instant)

Étape 1 — Initialiser le dépôt local

Dans le dossier de ton projet :

bash

```
git init
```

Ça crée un dossier caché `.git/` qui permet à Git de suivre ton projet.

Étape 2 — Ajouter les fichiers au suivi

bash

```
git add .           # ajoute TOUS les fichiers du dossier
git add index.html  # ou un fichier spécifique seulement
```

`git add` place les fichiers dans la **zone de staging** (prêts à être committés).

Étape 3 — Créer le premier commit

bash

```
git commit -m "Initial commit"
```

C'est une photo de l'état actuel de ton projet, avec un message descriptif.

Étape 4 — Lier au dépôt GitHub distant

Copie l'URL de ton dépôt GitHub (format `https://github.com/ton-user/ton-repo.git`), puis :

bash

```
git remote add origin https://github.com/ton-user/ton-repo.git
```

Pour vérifier que la liaison est bien faite :

bash

```
git remote -v
```

Étape 5 — Envoyer sur GitHub

bash

```
git push -u origin main
```

L'option `-u` définit `origin/main` comme branche distante par défaut. Les prochains push seront simplement `git push`.

> **Si ta branche s'appelle `master`** (anciens projets) : remplace `main` par `master`.

Workflow pour les modifications suivantes

Une fois la liaison en place, le cycle classique est :

bash

```
git add .
git commit -m "Description des changements"
git push
```

- `git add .` → **sélectionne** les fichiers à sauvegarder
- `git commit` → **crée la sauvegarde** en local
- `git push` → **envoie** cette sauvegarde sur GitHub

Créer une second prod pour séparer dév et production

Créer et basculer sur la branche dev

bash

```
git branch develop        # crée la branche
git checkout develop      # bascule dessus
```

Ou en une seule commande :

bash

```
git checkout -b develop
```

Envoyer la branche sur GitHub

bash

```
git push -u origin develop
```

Cycle de travail quotidien

bash

```
git checkout develop      # toujours vérifier qu'on est sur develop
# ... tu codes ...
git add .
git commit -m "feature: ajout section contact"
git push
```

Quand une feature est prête → fusionner sur master

bash

```
git checkout master       # revenir sur master
git merge develop         # fusionner develop dans master
git push                  # envoyer la prod à jour sur GitHub
```

Voir sur quelle branche tu es

bash

```
git branch                # liste les branches, * = branche active
```

Schéma mental

```
master   →  version stable, ce que les gens voient
develop  →  ton bac à sable, tu codes ici
```

> En entreprise tu verras souvent aussi des branches `feature/nom-fonctionnalité` créées depuis `develop` pour isoler chaque tâche, puis fusionnées dans `develop` une fois terminées.

Bonus:

Commandes utiles en bonus

Commande	Utilité
`git status`	Voir les fichiers modifiés / en staging

Commande	Utilité
`git log --oneline`	Historique des commits
`git pull`	Récupérer les changements depuis GitHub
`git branch -M main`	Renommer la branche locale en `main`

Récupérer depuis GitHub → local

bash

`git pull origin master`

Télécharge + fusionne les changements distants dans ton dossier local.

Pour la connexion demander au `git push -u origin main`

GitHub n'accepte plus les mots de passe depuis 2021. Il faut utiliser un **token d'accès personnel** à la place.

Créer un token sur GitHub

1. Va sur **github.com** → **Settings** (ton avatar en haut à droite)
2. **Developer settings** (tout en bas du menu gauche)
3. **Personal access tokens** → **Tokens (classic)**
4. **Generate new token (classic)**
5. Coche la case `repo` (accès complet aux dépôts)
6. Clique **Generate token** et **copie-le immédiatement** (il ne s'affiche qu'une fois)

Utiliser le token

Relance le push, et quand il demande le mot de passe, **colle ton token** à la place :

bash

```
git push -u origin master
```

```
Username: duckzla
Password: ghp_xxxxxxxxxxxxxxxxxxxx   ← ton token ici
```

Éviter de le retaper à chaque fois

bash

```
git config --global credential.helper store
```

Après le prochain push réussi, Git mémorisera le token automatiquement.